

The first potential solution we'll discuss is the simplest and has the worst performance. After that, we'll discuss solutions that trade a little complexity for a lot of performance improvement.

Brains vs. Brawn

It might occur to you to make a list of every possible itinerary that can connect our cities and evaluate them one by one to see which is best. If we want to visit three cities, the following is an exhaustive list of every order in which they can be visited:

- 1, 2, 3
- 1, 3, 2
- 2, 3, 1
- 2, 1, 3
- 3, 1, 2
- 3, 2, 1

It shouldn't take long to evaluate which is best by measuring each of the lengths one by one and comparing what we find for each of them. This is called a *brute force* solution. It refers not to physical force, but to the effort of checking an exhaustive list by using the brawn of our CPUs rather than the brains of an algorithm designer, who could find a more elegant approach with a quicker runtime.

Sometimes a brute force solution is exactly the right approach. They tend to be easy to write code for, and they work reliably. Their major weakness is their runtime, which is never better and usually much worse than algorithmic solutions.

In the case of the TSP, the required runtime grows far too fast for a brute force solution to be practical for any number of cities higher than about 20. To see this, consider the following argument about how many possible itineraries there are to check if we are working with four cities and trying to find every possible order of visiting them: